LAB5 - Group 10
Project “Maze game” report

Students: Nadine Tawil, Mohamed Salah Ahmed, Victor U. Honkanen, Rawan Saleh

---

1. Introduction

This project implements an interactive maze game using a Raspberry Pi as the main controller, combined with physical input/output devices (sensors/actuators) and a visual dashboard built with Node-RED. The game allows the player to navigate through an 8×8 maze grid from a defined start position to a finish position. The objective of the game is counting how many steps the player takes to get to the finish point. The player interacts with the system using a joystick and a push button, and receives real-time feedback through LEDs, a buzzer, an LCD screen, and a Node-RED dashboard. The project demonstrates key concepts in embedded systems and Internet of Things (IoT) design, including sensor interfacing, actuator control, state-based game logic, and message-based communication using the MQTT protocol. By integrating Python code running on the Raspberry Pi with Node-RED flows, the system provides both physical and graphical representations of the maze state.

2. System overview

The system consists of the following main elements:
1.Raspberry Pi: running a Python-based maze game
2. Input devices (sensors): analog joystick and push button
3. Output devices (actuators): RGB LED, buzzer, and I2C LCD display
4. Communication layer: MQTT broker running locally on the Raspberry Pi
5. Visualization layer: Node-RED flow that subscribes to MQTT topics and displays the maze state and movement information
The Raspberry Pi reads user input from the joystick and button, updates the game state according to the maze rules, and publishes relevant information (current position and joystick movement) via MQTT. Node-RED subscribes to these topics and visualizes the maze and player position in real time.

3. **Hardware setup and components**

Raspberry Pi (controller)

The Raspberry Pi acts as the central processing unit of the system. It runs the Python script that contains the maze logic, handles GPIO input/output, and manages MQTT communication.

Input/output devices

1. Joystick (analog input/sensor)

The joystick provides directional input (up, down, left, right). It is connected through an external ADC (I2C-based), since the Raspberry Pi does not have native analog inputs. The joystick's X and Y axes are read via the I2C bus, converted into digital values, and interpreted as movement commands.

2. Push button (Digital input/sensor)

The push button is connected to a GPIO pin. When pressed, it triggers a special action in the game: requesting a hint. Hints are limited to a maximum number to preserve the challenge of the game.

3. RGB LED (actuator)

The RGB LED provides immediate visual feedback: blue for game start or hint indication, green for valid move and red for collision with a wall or invalid action

4. Buzzer (actuator)

The buzzer provides audible feedback for events such as hitting a wall or confirming a valid move, which enhances user interaction by adding an audio cue alongside visual signals.

5. LCD Display (actuator)

A 16×2 character LCD connected via I2C displays game messages such as the number of steps taken, wall collisions, hints, and the finish message.

4. Game Logic and Interaction Design

- **Maze representation and states**
  The maze is modeled as an 8×8 grid stored as a matrix, where a value of **1** represents a free path, while **0** denotes a wall. The player's position within the maze is encoded using a concise state notation (for example, a73), where the numeric indices correspond to the current row and column. The game begins at a predefined start state (a73) and concludes when the player successfully reaches the finish state (a03).
- **Movement handling and core functions**
  Player movement is governed by two key functions. The possible_moves() function evaluates the current position and determines which directions are legally accessible based on the maze structure. When a movement input is provided, the next_state() function computes and updates the player's position accordingly, ensuring that state transitions remain consistent with the maze rules.
- **Hint system and invalid moves**
  If the player attempts an invalid move, such as trying to enter a wall, the system immediately responds with visual and audio feedback, while the player's position remains unchanged. Additionally, a hint mechanism is implemented through a push button. When pressed, the system displays all valid movement options from the current position on the LCD. To maintain gameplay challenge the number of available hints is deliberately limited.
- **Feedback and User Interaction**
  Every player action triggers coordinated, multimodal feedback designed to enhance usability and engagement. The LCD provides clear textual status updates, RGB LEDs convey visual cues corresponding to different game events, and the buzzer delivers immediate audio confirmation. This combination of visual, auditory, and textual feedback creates an intuitive and immersive user experience, reinforcing correct actions and clearly signaling errors or special events.

5. MQTT communication and Node-RED integration and visualization

The system uses MQTT for lightweight, publish–subscribe communication using the following topics:

- maze/state: publishes the current maze position (ex.a63)
- joystick/move: publishes the detected joystick direction

The Python script on Raspberry Pi acts as an MQTT publisher. After each move, it publishes the updated maze state and the joystick direction. This decouples the game logic from the visualization layer.

Node-RED subscribes to the MQTT topics and processes incoming messages with a flow that includes:

- MQTT input nodes for maze/state and joystick/move topics.
- Debug nodes for monitoring messages
- Dashboard elements for displaying position, direction, and progress
- Maze visualization function that maintains an 8×8 symbolic representation of the maze using X for walls, S for the start position and F for the finish position. These symbols are mapped to colored emoji to create a clear, intuitive visual display. The current player position (received via MQTT, ex. a63) is highlighted in red. Each update rebuilds the table and sends it to the Node-RED dashboard, allowing real-time tracking of the player's movement through the maze.

6. Overall system operation

- Sensors (joystick and button) sense the physical changes caused by the player and send signals to the controller.
- The controller (Raspberry Pi) reads the input and determines the intended action using Python game logic that validates the move and updates the game state.
- Actuators (LED, buzzer, and LCD) provide feedback.
- MQTT publishes the updated state and movement which are received by Node-RED that updates the visual maze and indicators.

7. Conclusion

In this project we successfully include 5 different devices (sensors, actuators), combined with Python-based control logic, MQTT communication, and Node-RED visualization to create an interactive maze game. Our project demonstrates how physical user interfaces can be tightly integrated with IoT dashboards, and highlights important concepts such as state machines, event-driven programming, and publish–subscribe communication. The modular design allows the system to be easily extended, for example by adding score tracking, remote control, or web-based interaction.